
Wherehouse

서울 1인 가구 주거지 추천 서비스
기획부터 배포까지 백엔드 1인 개발

https://github.com/bumjinDev/wherehouse_SpringBoot

배포 서비스: warehouse.it.kr

서비스 기획

비즈니스 문제

MZ 1인 가구 750만 시대, "안전하고 살기 좋은 동네"를 종합 비교할 서비스가 없다

핵심 요구사항

"정량적 공공데이터만으로는 반영할 수 없는 실제 거주 경험을 어떻게 추천 시스템에 반영할 것인가?"

주거지 추천 서비스에 필요한 것

요구사항	사용자 니즈	Wherehouse 접근
종합적 데이터	가격뿐 아니라 안전·편의시설까지 한번에 비교.	공공데이터와 카카오 맵 API 반영
실거주 경험	실제 살아본 사람의 체감 정보를 추천 결과에 반영 필요	리뷰 시스템 + 키워드 추출 + 하이브리드 점수 통합
가능한 매물 조회	데이터 근거가 있는 추천 결과를 원한다	Fallback 재 검색 로직 + 가중치 적용

기존 부동산 서비스

직방/다방 매물 중심 → 가격만 비교 가능

네이버부동산 실거래가 중심 → 지역 생활 정보 부족

Wherehouse

기획 1. 생활 인프라 종합 비교 + 실거주자 리뷰 통합

기획 2. 살기 좋은 동네" 관점의 데이터 기반 추천

핵심 기능

기능 1. 조건별 주거지 추천 $\text{가격} \times \text{안전성} \times \text{편의시설}$ 종합 분석

기능 2. 상세 지도 분석 선택한 위치의 편의성과 안전성 분석

기능 3. 실거주자 리뷰 반영 공공데이터와 실거주 경험을 모두 고려

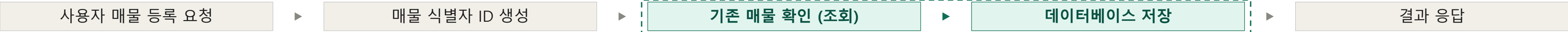




사용자 매물 중복 등록 방지

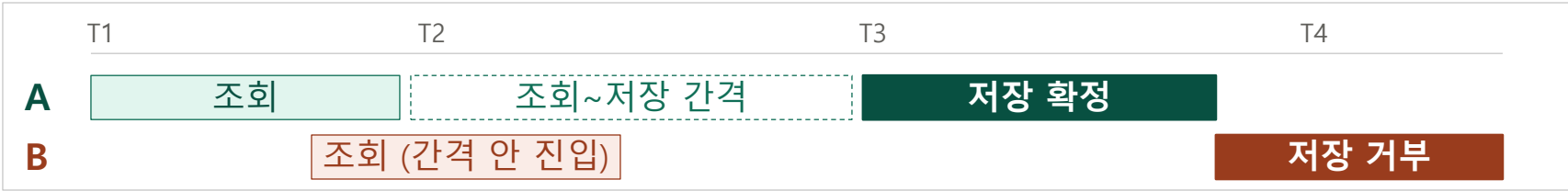
문제 정의 — 서로 다른 두 사용자가 같은 매물(시군구·지번·아파트명·층·전용면적 동일)을 같은 시점에 등록 시 한 사용자 등록 성공, 다른 사용자 중복 매물 거부 응답

의사결정 지점 — 확인 후 저장 구간



동시 등록 발생 상황

두 사용자가 같은 매물을 동시에 등록 요청 시, 두 요청 모두 매물 조회 단계에서 '미등록' 응답을 받은 뒤 저장 단계로 진입한다.



후보 선정과 측정 의도

정합성을 보장 가능한 두 후보인 애플리케이션 임계 구역과 데이터베이스 키 제약을 실측 비교

동일 매물 5건 동시 등록

동일 매물 동시 등록 시 정확히 하나만 성공하는가 확인.

서로 다른 매물 5건 동시 등록

서로 다른 매물 등록 요청이 병렬 처리되는가 확인. 비즈니스적 순서 의존성 없음.

혼합 10건 (동일 매물 2건 + 서로 다른 매물 8건)

운영 상황에 가까운 혼합 부하. 충돌 요청만 직렬화되고 충돌 없는 요청은 병렬 처리되는가 확인.

애플리케이션 임계 구역 — 인스턴스 단위 단일 락

U1U2

단일 락 게이트

→ 한 명 통과 / 나머지 대기 (매물 종류 무관)

U3

데이터베이스 키 제약 — 매물 ID 단위

U1 (매물 A)U2 (매물 A)

매물 A 락

→ U1·U2만 경합 (매물 A끼리)

U3 (매물 C)

매물 C 락 (독립)

정량 측정 결과

시나리오	애플리케이션 임계 구역	데이터베이스 키 단위 락	사용자 영향 차이
동일 매물 5건	1+4 (135~138ms)	1+4 (149~153ms)	두 후보 동일
다른 매물 5건	21→46ms (+25ms 계단식)	20~21ms (편차 1ms)	애플리케이션 측만 +25ms 대기
혼합 10건 (동일 2+다른 8)	23→70ms (+47ms 계단식)	24~26ms 병렬, 동일 매물만 +8ms	애플리케이션 측만 +39ms 대기

두 후보 모두 동일 매물 등록에서는 정합성이 동등하지만, 다른 매물과 혼합 부하에서 애플리케이션 임계 구역은 충돌 없는 등록까지 대기시키고 데이터베이스 키 단위 락은 동일 매물 충돌만 대기시킨다.

결정

데이터베이스 키 단위 락

(매물 ID 단위)

근거 1 — 정합성

두 후보 모두 정합성 보장 (동일 매물 1건 정상·나머지 거부). 차이는 다른 매물 / 혼합 부하에서 드러남.

근거 2 — 비즈니스 충돌 범위

동일 매물 등록 요청 사이에서만 비즈니스 충돌 발생. 서로 다른 매물은 비즈니스적 순서 의존성 없음.

근거 3 — 일치성 (결정 사유)

데이터베이스 키 단위 락은 동일 매물 등록 요청끼리만 경합 = 비즈니스 충돌 범위와 일치.

애플리케이션 측 락은 모든 등록 요청을 직렬화.

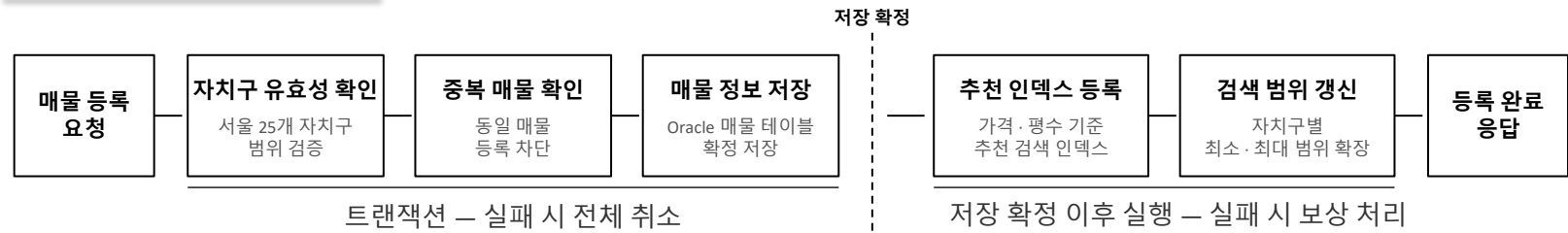
Redis 정합성 보장

매물 등록 후 추천 검색 누락 · 유령 데이터 발생을 막기 위한 확정 이후 Redis 동기화 설계

문제 진단

매물 등록 시 Oracle 저장과 Redis 추천 인덱스 반영이 별도의 트랜잭션으로 분리되면,목록에는 확인 되나 추천 검색에는 누락되거나, DB에는 없는 Redis 유령 데이터 문제 발생

기능의 서비스 위치



아키텍처 설계 근거

영속성 반영 전 인메모리 반영은 단순 트랜잭션 정합성을 고려하지 않으면 부분 실패 시 유령 데이터가 남을 수 있다고 예상.

테스트 내용	Oracle	인메모리 주거지 추천 정보	사용자 영향
주거지 정보 반영 전체 실패 /트랜잭션 없음	저장	전체 미 반영	추천 검색 누락 및 매물 재 등록 불가능
주거지 정보 반영 전체 실패 /트랜잭션 있음	미 저장	전체 미 반영	저장 실패 및 정합성 보장
주거지 정보 반영 부분 실패 /트랜잭션 없음	저장	매물 데이터 반영 / 매물 검색 데이터 미 반영	매물 정보 확인 가능하나 추천 기능 이용 불가
주거지 정보 반영 부분 실패 /트랜잭션 있음	미 저장	매물 데이터 반영 / 매물 검색 데이터 미 반영	매물 정보 미 등록 및 추천 정보만 확인

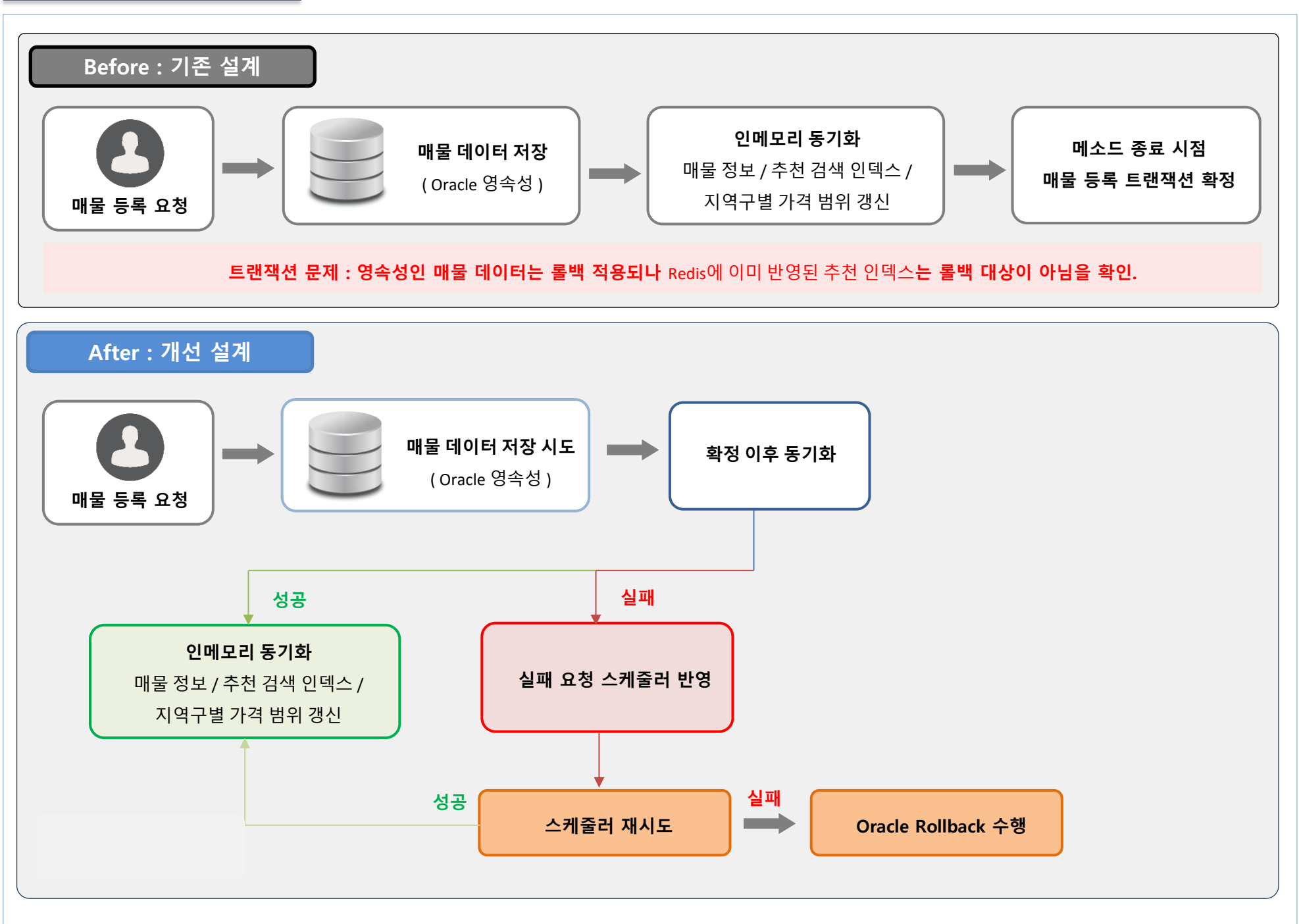
영속성 보장과 인메모리 반영은 단순 트랜잭션 정책만으로 해결되지 않으며, 부분 실패 시 유령 데이터가 남을 수 있음을 확인함.

설계 판단

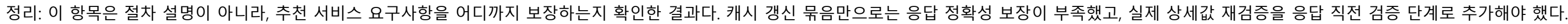
문제 매물 저장(Oracle)과 추천 반영(Redis)은 서로 다른 저장소이므로, 한쪽만 실패하면 사용자에게 불일치가 노출된다.

- ① 추천 반영 시점
- “Oracle 저장 확정 전 Redis에 먼저 반영되면 롤백 시 유령 데이터가 남을 수 있으므로, 저장 확정 이후 Redis 동기화로 순서를 조정했다
- ② 실패 보상 전략
- 실패 이력 기록 후 스케줄러 자동 재시도를 설계하여 일시 장애는 자동 복구, 일정 횟수 이상 초과 시 등록 철회로 누락 방지 방지
- ③ 실패 기록 보호
- 실패 기록 저장을 별도 트랜잭션으로 분리하여 실패 기록하여 일시적인 저장 실패가 정상 매물 등록을 취소시키지 않도록 차단

아키텍처 개선 전/후 비교



문제 정의 — 사용자의 매물 추천 요청과 다른 사용자의 매물 가격 수정이 동시 발생 시, 사용자가 요청한 예산 범위를 초과하는 매물이 추천 응답에 포함된다.



→ 채택 안의 차단 범위가 비즈니스 충돌 범위 전체와 일치.

방문 예약 슬롯 동시성 제어 전략 결정

문제 정의 — 한 슬롯에 여러 사람이 같은 순간 예약을 시도해도, 정확히 한 명만 확정되고 나머지는 정상적으로 거부되어야 한다. 그 책임을 어느 계층에 둘 것인가.

요청 흐름과 계층 — 동시성 제어가 놓이는 위치



후보 4종과 측정 설계

① 무제어
조건 없이 갱신 — 모두 통과

② 조건부 UPDATE
갱신할 때 예약 가능 상태를 함께 확인
채택

③ 비관적 락
조회하는 순간 슬롯을 잠금

④ 낙관적 락
커밋 시점에 변경 충돌을 감지

측정 — 다섯 요청을 한 슬롯에 동시 집중. 남은 데이터 · 단계별 처리 로그 · 실제 쿼리 소요시간 · 사용자 응답을 교차로 확인

정량 측정 결과 — 4종 교차 비교

후보	정합성 (슬롯당 확정 수)	차단 위치	거부 처리 비용
① 무제어	확정 5건 · 중복	없음	예약 5건 모두 등록
② 조건부 UPDATE 채택	1건	쓰기 단계에서 차단	없음
③ 비관적 락	1건	읽기 단계에서 대기 (73~128ms)	없음
④ 낙관적 락	1건	커밋 단계에서 충돌 감지	예약까지 진행 후 롤백

① 무제어 시 한 슬롯에 확정 5건이 남지만 사용자 응답은 모두 성공이다.

정합성은 ②③④ 모두 확정 1건으로 동일하다 — 채택을 가른 것은 거부되는 요청을 어느 단계에서, 어떤 비용으로 처리하는가다.

결정

조건부 UPDATE

예약 가능 확인과 점유를 한 번에 처리해 경합 구간 자체를 없앤다

근거 1 — 거부 비용

거부 요청이 예약 등록 전에 차단되어 불필요한 작업이 없다 (④는 예약까지 진행한 뒤 취소)

근거 2 — 짧은 잠금 구간

점유(쓰기) 시점에만 잠가 잠금 구간이 짧다 — ③은 읽기부터 커밋까지 잠금을 유지한다

DB 조회 구조 개선 — 반복 집계 조회 (N+1)

문제 정의 — 전세 추천 시 25개 자치구를 순회하며 자치구별 findAllById() 로 리뷰 통계를 개별 조회. 요청당 최대 25회 RDB 호출 + 6 커넥션 풀에 20 동시요청 경합.

의사결정 지점 — 리뷰 통계 RDB 조회 방식

요청

사용자 추천 요청

▶

후보 매물 수집

자치구별 후보 매물 ID 수집

▶

조회 방식 · 의사결정

리뷰 통계 RDB 조회
N+1 → Bulk → Chunk

▶

점수 계산

매물 점수 계산

▶

응답

추천 응답

접근 후보 3종과 측정 의도

① N+1 (원형)
자치구별 개별 호출 — 반복 327회·Hard Parse

② Bulk Fetch
전체 ID 병합 1회 — 호출 −93.9%, 단 가변 IN절·OR분해

③ Chunk 분할 (채택: 61)
고정 크기 분할 — 1000·22·61 실측으로 균형점 탐색

채택

정량 측정 결과 — 측정이 다음 결정을 이끈 과정 (5개 구성 · 원시 로그는 별첨)

	① 문제 출발점	② ▶ 반복 호출 제거	③ ▶ SQL 텍스트 고정	④ ▶ Chunk 세분화	⑤ ▶ 경합 억제 · 균형
측정 항목	1차 · N+1	2차 · Bulk	3차 · Chunk 1000	3차 · Chunk 22	3차 · Chunk 61
RDB 호출 (20req)	327회	20회	21회	381회	144회
SQL_ID 종류	78개	20개	20개	11개	15개
대표 SQL 실행 (재사용)	1~35 (분산)	1 (재사용 0)	최대 2	341	115
실행계획 · Cost	INLIST · 6~594	INLIST · OR분해	Full Scan · 191~333	INLIST · 27	INLIST · 68
앱 RDB 시간 비율	41.3%	19.5%	17%	48.5%	17.6%
커넥션 풀 대기 (최대)	9	2	1	10	4 (심각 0)
판단 근거	호출·파싱 폭증	재사용 0 · OR분해	분할 미발동 · Full Scan	경합 폭증 (대기 10)	균형 → 채택

N+1 반복 호출을 Bulk로 제거했으나 가변 IN절이 Hard Parse를 남겼고, Chunk 고정으로 해소 — 크기 1000·22·61 실측으로 DBMS 최적화·호출·경합의 균형점 61을 확정.

결정

Chunk 61 분할 조회

고정 61개 IN절 · INLIST ITERATOR + INDEX UNIQUE SCAN

핵심 성과 — Soft Parse 88.7% · INLIST Cost 68 · 풀 대기 4(심각 0)

측정 계층: V\$SYSSTAT · V\$SQL · V\$SQL_PLAN · App Bottleneck Log · HikariCP

별첨: V\$SQL 전체 SQL_ID · 20스레드 원시 로그 · Plan Hash · Chunk별 상세 비교표

근거 1 — Library Cache 재사용

바인드 61 고정으로 대표 SQL 115회 재사용(Soft Parse 88.7%) — Bulk·1000은 재사용 실패.

근거 2 — 실행계획 효율

전환 경계(54~276) 아래로 INLIST ITERATOR + INDEX UNIQUE SCAN(Cost 68) 확보 — 1000은 Full Scan.

근거 3 — 호출수·경합 균형 (결정 사유)

Chunk 22는 호출 381·대기 10으로 과도 → 61은 144·대기 4(심각 0)로 균형 채택.

Redis 호출의 RTT 직렬 문제 해결

조건 조회 75회 순차 호출에서 25회 일괄 호출 변환

설계

기능의 서비스 내 위치



Pipeline 미 적용 - TCP 흐름



Pipeline 미적용 구간 TCP 흐름 실측 확인

- 관측 1 : 단일 조건 Latency 중 I/O 비율 99.71%
- 관측 2 : 3개 조건 Latency 31 : 36 : 33

자치구당 RTT 3회 누적 문제

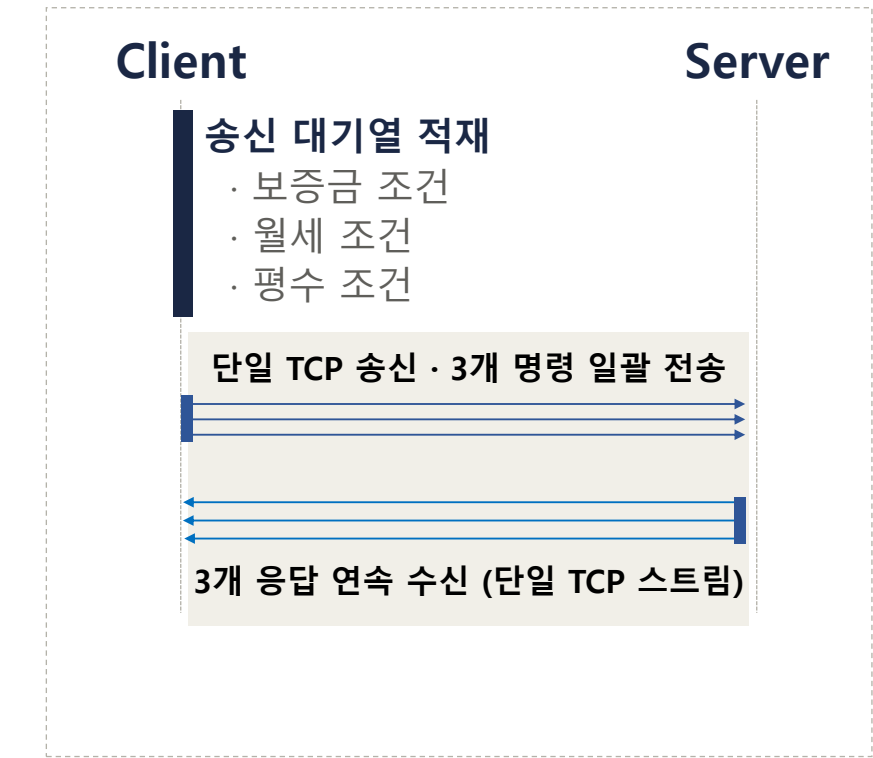
25개 자치구 전체 응답 시간 (합계) 6,540 ms	자치구당 평균 응답 시간 (강남구 제외) 128.95 ms	I/O 비율 99.71 %
-----------------------------------	-------------------------------------	-------------------

해결 방안

원인

직렬화 원인은 Redis·TCP 자체 제약이 아닌 동기 호출한 결과

Pipeline 적용 시 TCP 흐름



Pipeline 적용 구간 TCP 흐름 실측 확인

- 관측 1 : RTT 1회 왕복 비용 변화 없음
39.3 ms ~ 38.5 ms (실측 차이 0.8 ms)
- 관측 2 : 평균 응답 절감 70.15 ms
(어플리케이션 계층 실측 응답 시간)

측정 근거 데이터 (관측 정량 근거)

계층	지표	미적용	적용	변화
어플리케이션	자치구 당 평균 응답 (강남구 제외)	128.95 ms	58.80 ms	-54.4%
어플리케이션	I/O 비율	99.71 %	99.62 %	유지
네트워크	단일 왕복 Latency	38.5 ms	39.3 ms	+0.8 ms
네트워크	자치구당 총 RTT (강남구 제외)	115.5 ms	38.9 ms	-66.3%

- 강남구는 응답 payload 4.0 MB가 병목 본질로 Pipeline 효과 제한적 (MethodTime -15.6%)

외부 API 호출 구조 문제 해결

상세지도 서비스에서 편의시설 정보를 불러오는 데 1.3초 이상이 소요되어 사용자 이탈 위험이 있었다. API 순차 호출을 병렬 처리로 전환.

문제 진단

부하 측정과 기능별 내부 로깅을 통해 외부 API 호출 단계가 전체 응답의 92.2%를 점유하는 병목으로 식별. (편의시설 15개 카테고리 순차 호출)



기술 의사결정

- 문제

15개의 각 카테고리를 카카오 맵 API로 동기 순차 호출하여, 외부 응답 대기가 선형 누적되는 구조.
- 설계 결정

동기 순차 호출에서 비동기 병렬 호출로 전환

1. CompletableFuture — 비동기 작업 합성·완료 대기·예외 격리를 단일 API로 제공

2. ThreadPoolExecutor — 풀 크기 기준을 동시 외부 호출 수로 설정

ForkJoinPool : CPU 연산의 분할·재분배에 최적화된 풀 구조로, 외부 응답 대기의 작업의 동시 호출 수 요건과 불일치

성과

항목	Before	After	개선율
편의시설 API 호출 구간 포함한 동작 전체 (Application 계층)			
평균	1,221ms	255ms	-79.1%
최대	1,651ms	1,103ms	-33.2%
편의시설 API 호출 구간			
평균	1,107ms	144ms	-87.0%
p95	1,192.5ms	240.5ms	-79.8%

매물 검색 쿼리 최적화

양방향 LIKE 패턴의 Full Table Scan을 전방 일치 + B-Tree 인덱스로 전환

진단

기능의 서비스 내 위치

